The Windows Event Logs and Forensics Playbook

A complete, detailed guide to Windows Event Logs, Sysmon, and host-based forensic investigation

From Event ID 4624 to a full attack timeline, this playbook builds the artifact knowledge and query fluency that SOC analysts and incident responders rely on to reconstruct what happened on a host.

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 4, 2026

Preface

Network logs tell you traffic happened; Windows Event Logs tell you what a *host* actually did — who logged in, what process ran, what file changed, what registry key got written. For most intrusions, the host is where the real story is: initial access, persistence, privilege escalation, and lateral movement all leave a trail across a handful of well-known Event IDs, if you know where to look.

This playbook is organized the way a real investigation unfolds: starting with the event log architecture itself, then the specific Event IDs that matter for authentication, process execution, and object access, then Sysmon and PowerShell logging (which cover what the built-in Security log often misses), then the broader universe of forensic artifacts — registry hives, Prefetch, Amcache, ShimCache — and finally how to pull all of it into a single timeline during an actual investigation.

What this book covers

- Windows Event Log architecture: channels, .evtx files, Event Viewer
- Key Security log Event IDs, organized by what they tell you
- Logon types and authentication event analysis
- Process creation logging and Sysmon's extended visibility
- PowerShell logging: script block, module, and transcription logging
- Object access and file system auditing
- Core forensic artifacts beyond the event log: registry, Prefetch, Amcache, ShimCache, LNK files
- Building an investigation timeline from multiple artifact sources
- Querying event logs from the command line and PowerShell
- Practical investigation recipes for common scenarios
- Recognizing log tampering and anti-forensic techniques

Swastik Sagar

Contents

1

Introduction to Windows Event Logs

1.1 Why host logs matter

A SIEM alert or a Wireshark capture can tell you traffic crossed the wire, but only the host itself can say what actually happened locally: which account logged in, which process spawned which child process, which file got written to disk. Windows Event Logs are the built-in mechanism for recording exactly that, and they're present on every Windows host by default, with zero extra tooling required to start collecting them.

1.2 The three core logs

- **Security** — authentication, authorization, and object access events; the log most relevant to investigations, and the most heavily audited
- **System** — OS-level events: service start/stop, driver issues, system time changes
- **Application** — events logged by installed applications, quality varies enormously by vendor

Auditing must be enabled

Windows does not log everything by default — many of the Security event types this book covers only appear if the relevant **audit policy** category is turned on (via Local Security Policy or Group Policy). An unconfigured host can look deceptively quiet simply because auditing was never enabled, not because nothing happened.

1.3 Where logs live

Default .evtx file locations

```
C:\Windows\System32\winevt\Logs\Security.evtx C:\Windows\System32\winevt\Logs\
System.evtx C:\Windows\System32\winevt\Logs\Application.evtx
```

Each log is stored as a binary `.evtx` file, viewable in Event Viewer, parsable with PowerShell, or exportable for ingestion into a SIEM.

2

Event Log Architecture

2.1 Anatomy of an event

Field	Meaning
Event ID	Numeric identifier for the event type, e.g. 4624 for a successful logon
Level	Information, Warning, Error, Critical
Provider/Source	Which component logged the event
Task Category	Sub-classification within the provider
Keywords	Tags like Audit Success / Audit Failure
Time Created	Timestamp, normally in UTC internally
Computer	Hostname that generated the event
Event Data	The event-specific fields (varies per Event ID)

2.2 Channels

Beyond the three core logs, Windows organizes many events into dedicated **channels** — for example `Microsoft-Windows-PowerShell/Operational` or `Microsoft-Windows-Sysmon/Operational` — visible in Event Viewer under *Applications and Services Logs*, each with its own retention and size settings.

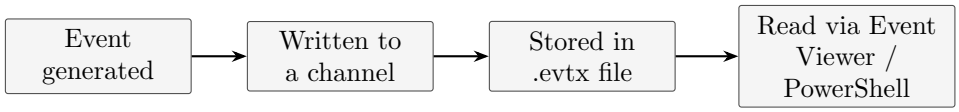

Figure 2.1. *Every Windows event follows this same path from generation to something an analyst can read.*

2.3 Log rotation and retention

Each log has a maximum size and a rotation policy (overwrite oldest first, archive, or never overwrite). On a busy domain controller, the Security log can rotate in hours; forwarding events to a SIEM promptly is the only reliable way to retain them long-term.

3

Key Security Event IDs

3.1 Authentication events

Event ID	Meaning
4624	Successful logon
4625	Failed logon
4634 / 4647	Logoff (system-initiated / user-initiated)
4648	Logon using explicit credentials (runas, or a service using different creds)
4672	Special privileges assigned to a new logon (admin-equivalent)
4768 / 4769	Kerberos TGT / service ticket requested
4776	NTLM credential validation attempted

3.2 Account and group management

Event ID	Meaning
4720	New user account created
4722	User account enabled
4724	Password reset attempt
4728 / 4732 / 4756	Member added to a global/local/universal group
4738	User account changed

3.3 Process and object events

Event ID	Meaning
4688	New process created
4689	Process exited
4656	Handle requested to an object
4663	Attempt to access an object (needs a matching SACL)
5140 / 5145	Network share accessed (5145 includes per-file detail)

3.4 Policy and log events

Event ID	Meaning
4719	System audit policy changed
1102	Security log cleared — a major red flag on its own
4697	A service was installed on the system
7045	New service installed (System log)

The one Event ID to always alert on

Event 1102 (audit log cleared) should essentially always trigger a review — legitimate administrators clear logs rarely, and it's a standard anti-forensic step for anyone trying to cover their tracks after gaining access.

4

Logon Types and Authentication Analysis

4.1 Logon type values

Event 4624 and 4625 both include a **Logon Type** field that reveals exactly how the authentication happened — often more useful than the success/failure result alone.

Type	Meaning
2	Interactive — physical console logon
3	Network — e.g. accessing a share, most remote authentication
4	Batch — scheduled task
5	Service — a service starting with configured credentials
7	Unlock — workstation unlocked
8	NetworkCleartext — credentials sent in cleartext over the network
9	NewCredentials — `runas /netonly`
10	RemoteInteractive — RDP
11	CachedInteractive — logon using cached domain credentials

Why logon type matters for detection

A Logon Type 10 (RDP) from an account that never uses RDP, or a Logon Type 3 (network) authentication against a workstation that has no business being accessed remotely, are both far more suspicious than the bare "successful logon" framing suggests — the type field is where the real signal usually is.

4.2 Correlating 4625 for brute force

- Many 4625 events for one account, from one or many source addresses, in a short window
- Many 4625 events for many different accounts from one source (spraying)
- A run of 4625 events immediately followed by a 4624 success from the same source

4.3 The Sub Status code

4625 events include a **Sub Status** code narrowing down exactly why the logon failed — `0xC000006A` is a bad password (account exists, password wrong), `0xC0000064` is a username that doesn't exist at all, and `0xC0000234` means the account is currently locked out — each tells a different story about what an attacker is actually doing.

5

Process Creation and Sysmon

5.1 Event 4688: process creation

With the right audit policy enabled, 4688 logs every new process, its command line, and its parent process — the single most useful event for reconstructing what actually executed on a host.

Command line auditing isn't automatic

Full command-line logging in 4688 requires a separate policy (*Include command line in process creation events*) beyond just enabling process-creation auditing — without it, you get the process name but not its arguments, which is often the part that actually matters.

5.2 Why Sysmon

Sysmon (System Monitor, a free Microsoft Sysinternals tool) extends far past what 4688 alone provides: process hashes, network connections made by a process, file creation events, registry modifications, named pipe activity, and process access events (like one process opening a handle to LSASS, a classic credential-dumping precursor).

Event ID	Meaning
1	Process creation (with hash and full command line)
3	Network connection
7	Image/DLL loaded
8	CreateRemoteThread — classic injection technique
10	Process access — e.g. a handle opened to LSASS
11	File created
12/13/14	Registry object added/modified/renamed
22	DNS query

5.3 The process creation tree

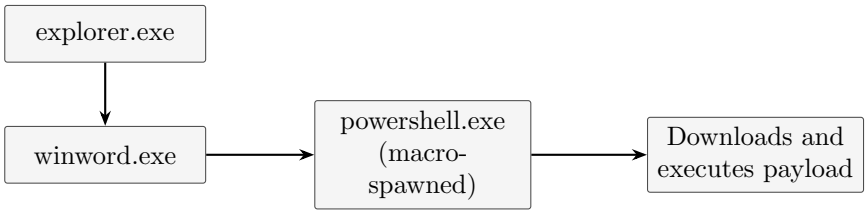

Figure 5.1. *A textbook malicious process tree: an Office app spawning a shell is a strong anomaly signal on its own, before even looking at what the shell did.*

Parent-child anomalies

`winword.exe` or `excel.exe` spawning `cmd.exe`, `powershell.exe`, or `wscript.exe` is one of the most reliable phishing-payload detection patterns available — legitimate document editing essentially never does this.

6

PowerShell Logging

6.1 Why PowerShell needs its own logging

PowerShell is both a legitimate administration tool and one of the most common post-exploitation frameworks, largely because it can execute entirely in memory — making file-based detection useless against it without the right logging enabled.

6.2 The three PowerShell logging types

Type	What it captures
Module logging	Logs pipeline execution details for specified modules
Script block logging	Logs the full text of scripts and commands as they execute, including de-obfuscated code
Transcription	Writes a full text transcript of a PowerShell session to disk

Script block logging is the most valuable

Attackers routinely obfuscate PowerShell with Base64 encoding or string concatenation — script block logging (Event ID 4104) captures the code *after* PowerShell's own engine has deobfuscated it internally, which is often the only way to see what a heavily obfuscated one-liner actually does.

6.3 Key PowerShell Event IDs

Event ID	Meaning
4103	Module logging: pipeline execution details
4104	Script block logging: full script content
400 / 403	Engine state started / stopped

Where PowerShell logs live

```
Applications and Services Logs > Microsoft > Windows > PowerShell > Operational
```

7

Object Access and File System Auditing

7.1 Enabling object access auditing

Object access events (4656, 4663) require both the audit policy enabled *and* a System Access Control List (SACL) configured on the specific file, folder, or registry key you want monitored — auditing is off by default even on files that already have restrictive permissions, since permissions and auditing are configured separately.

7.2 Reading a 4663 event

Field	**Meaning**
Subject	The account that performed the access
Object Name	Full path of the file/folder/key accessed
Access Mask	Numeric code for what kind of access (read, write, delete)
Process Name	Which executable performed the access

7.3 Share access

Event 5140 logs that a network share was accessed; 5145 goes further and logs per-file access checks within that share — useful for tracing exactly which files an attacker touched during lateral movement over SMB.

8

Forensic Artifacts Beyond the Event Log

8.1 Why other artifacts matter

Event logs can be cleared, disabled, or simply rotated out before an investigation begins — these other artifacts often persist independently and can corroborate or even replace missing log data.

8.2 Registry artifacts

Artifact	What it shows
Run / RunOnce keys	Common persistence locations, programs launched at logon
UserAssist	GUI-launched programs, with run count and last-run time
ShimCache (AppCompatCache)	Evidence a file existed and its path, even if later deleted
Amcache.hve	Detailed program execution evidence including file hashes
MRU lists	"Most recently used" file/folder access history

8.3 Filesystem and execution artifacts

Artifact	What it shows
Prefetch (`.pf` files)	Evidence a program executed, with run count and last 8 run timestamps
LNK files	Shortcuts auto-created when a file is opened, even from removable media
Jump Lists	Recently/frequently accessed files per application
$MFT / USN Journal	Master File Table and change journal — file creation, modification, deletion history
Recycle Bin ($I/$R files)	Deleted file metadata and content, if not yet purged

Prefetch is often the last artifact standing

Prefetch files survive many cleanup attempts because attackers frequently don't know they exist. A `.pf` file confirming a tool ran, with a timestamp, is often the single most durable piece of execution evidence on a Windows host.

9

Timeline Analysis

9.1 Why build a timeline at all

No single artifact tells the whole story — a timeline correlates event logs, filesystem timestamps, registry writes, and Prefetch data into one chronological view, which is usually what actually reveals the attack sequence rather than any individual log entry in isolation.

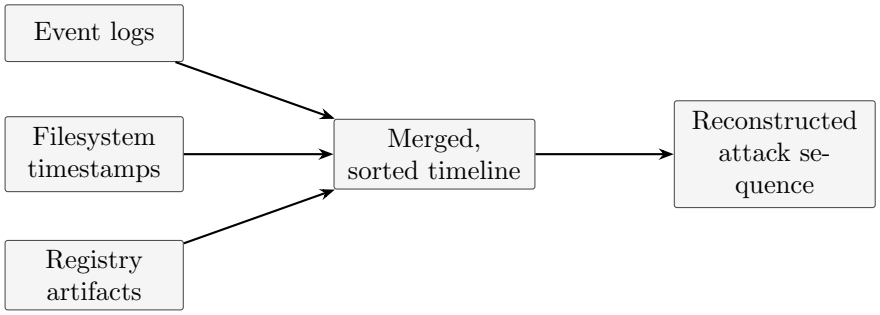

Figure 9.1. *Timeline analysis is fundamentally a merge operation across every artifact source that carries a timestamp.*

9.2 Common timeline tools

Tool	Role
Plaso / log2timeline	Parses many artifact types into one super-timeline
Timesketch	Web UI for collaboratively reviewing and annotating timelines
KAPE	Fast artifact collection, often feeding directly into Plaso
Eric Zimmerman's tools	Individual parsers for Prefetch, ShimCache, MFT, LNK files, and more

Timestamp caveats

Not every timestamp means what it looks like — file modification times can be altered (timestomping), and some artifacts record UTC while others record local time. Always confirm the timezone and be skeptical of timestamps that don't corroborate against at least one other source.

10

Querying Event Logs

10.1 PowerShell: Get-WinEvent

Basic queries

```
Get-WinEvent -LogName Security -MaxEvents 50
Get-WinEvent -FilterHashtable @ LogName = 'Security'; Id = 4625  -MaxEvents 100
```

Filtering by time range

```
Get-WinEvent -FilterHashtable @ LogName = 'Security'; Id = 4624; StartTime =
(Get-Date).AddHours(-24)
```

Reading an offline .evtx file

```
Get-WinEvent -Path "C:\Evidence\Security.evtx" -MaxEvents 20
```

10.2 Extracting specific fields

Pulling structured fields from event XML

```
Get-WinEvent -FilterHashtable @LogName='Security'; Id=4625 | ForEach-Object
[xml]$xml = $_.ToXml() $xml.Event.EventData.Data
```

10.3 wevtutil, the older command-line tool

wevtutil examples

```
wevtutil qe Security /c:20 /rd:true /f:text wevtutil epl Security C:\Evidence\
Security.evtx
```

`qe` queries events, `epl` exports a full log to a file — useful for evidence collection, since it preserves the original binary format rather than a reformatted export.

Prefer Get-WinEvent for analysis

`wevtutil` is reliable for bulk export, but `Get-WinEvent`'s `-FilterHashtable` and native XML access make it far more practical for actual field-level filtering and analysis work.

11

Practical Investigation Recipes

11.1 Suspected brute force against a local account

PowerShell

```
Get-WinEvent -FilterHashtable @LogName='Security'; Id=4625 | Group-Object
$_.Properties[19].Value | Sort-Object Count -Descending
```

Groups failed logons by source IP (the relevant property index varies by Windows version — verify against the raw XML first) to quickly spot a single source responsible for a spike in failures.

11.2 Confirming what a suspicious process actually did

- Pull the 4688 (or Sysmon Event ID 1) for the process, noting the full command line and parent process
- Check Sysmon Event ID 3 for any network connections made by that process ID
- Check Sysmon Event ID 11 for files it created
- Cross-reference Prefetch for confirmation the binary actually executed, with a timestamp

11.3 Investigating a compromised account

- Pull every 4624/4625 for the account across the retention window, noting Logon Type and source
- Check 4672 for any privilege escalation tied to that logon
- Check 4720/4728/4732 around the same window for account or group changes made using those credentials
- Correlate against VPN or firewall logs for the same source IP, if available

11.4 Checking for log tampering

Quick check for an audit log clear

```
Get-WinEvent -FilterHashtable @LogName='Security'; Id=1102
```

A hit here means someone cleared the Security log — treat this as a high-priority finding on its own, then check other logs (System, PowerShell operational, Sysmon) for what might be missing from Security during the same window.

12

Anti-Forensics and Log Tampering

12.1 Common anti-forensic techniques

- **Clearing the event log** — generates its own Event 1102, which is itself evidence
- **Disabling auditing mid-session** — Event 4719 (audit policy changed) flags this
- **Timestomping** — altering file MAC (Modified/Accessed/Created) timestamps to blend in with legitimate files
- **Living-off-the-land** — using built-in tools (PowerShell, WMI, certutil) instead of custom malware, to avoid signature-based detection
- **Disabling Sysmon or stopping the logging service** — creates a Service Control Manager event (7035/7036) worth watching for

12.2 Detecting a gap in the timeline

A suspiciously quiet period in an otherwise active log — especially one bounded by an audit policy change or service stop/start — is itself a finding, even without a single explicit malicious event inside the gap.

Redundant logging is the real defense

The best defense against log tampering isn't detecting it after the fact — it's forwarding logs to a central SIEM in near real time, so an attacker clearing the local `.evtx` file doesn't remove the copy that already left the host.